

Task 1 — TensorFlow Playground Experiment

Playground par jaake ye steps follow karo:

1. **Dataset:** "Two clusters" wala blue/orange dots dataset select karo (default hi theek hai)
2. **Architecture:** "+ Hidden Layer" se 1 hidden layer add karo, aur uske neurons ko 3 set karo (use the "+" "/" "-" buttons next to the layer)
3. **Learning rate experiment:**
 - **Low learning rate (e.g., 0.001 or 0.003):** Decision boundary bohot **slowly** move karega. Loss curve gradually decrease karta hai, smooth steps mein — convergence safe hai but **bohot time leta hai** (zyada epochs chahiye).
 - **High learning rate (e.g., 0.3 or 1):** Boundary **fast** shift hoga, but kabhi-kabhi loss curve **oscillate** karega ya overshoot ho jaayega — sometimes decision boundary instable ho jaata hai aur converge hi nahi karta (loss spikes up-down).
 - **Sweet spot (e.g., 0.03 - 0.1):** Generally fastest stable convergence milta hai — boundary smoothly orange/blue dots ke beech settle ho jaata hai.

Observation summary: Bohot low learning rate = slow but stable. Bohot high = fast but unstable/divergent. Moderate rate = best balance — yahi gradient descent ka core trade-off hai.

Task 2 — Biological vs Artificial Neuron Diagram

visualize

visualize show_widget

Mapping (color = correspondence):

- **Dendrites ↔ Inputs (x):** dono signal/data ko receive karte hain.
- **Soma ↔ Weighted sum + activation function:** soma signals ko combine karta hai, jaise artificial neuron weighted sum ($\sum w \cdot x + b$) calculate karke activation function (f) apply karta hai.
- **Axon ↔ Output (y):** dono processed signal ko aage forward karte hain — axon next neuron ko, artificial neuron next layer ko.
- **Weights (w):** biological neuron mein synaptic strength ke equivalent hain — ye decide karte hain ki kaunsa input zyada important hai.

Task 3 — Apps Using Deep Learning Daily

App	Feature	Deep Learning kaise help karta hai
Instagram	Feed	Deep learning models (recommendation neural networks) user ke past likes, watch-time, aur interactions ko analyze karke predict karte hain ki user ko aage kaunsa content pasand aayega — isse engagement aur relevance better hoti hai, har user ka feed personalized hota hai.
	recommendation / Explore page	
Spotify	"Discover Weekly" / song recommendations	Deep learning (collaborative filtering + audio embeddings ke through) songs ki audio features (tempo, genre, mood) aur listening patterns ko samajhta hai, isse aapko exact taste ke similar naye songs milte hain bina manually search kiye.
Zomato / Swiggy	Delivery time prediction / restaurant recommendation	Neural networks traffic, distance, restaurant prep-time aur historical delivery data ko combine karke accurate ETA predict karte hain — isse customer ko realistic delivery time milta hai aur trust badhta hai.

Task 4 — Workflow for a Deep Learning Movie Recommendation Model

Stage 1: Data

- User data collect karo: watch history, ratings, search queries, time spent on each title, demographic info (if available).
- Movie metadata collect karo: genre, cast, director, release year, plot tags.
- Data cleaning: missing values handle karo, duplicate entries remove karo, categorical features ko encode karo (e.g., genre → one-hot or embeddings).
- Train/validation/test split karo (e.g., 80/10/10).

Stage 2: Model

- Architecture choose karo — typically a **neural collaborative filtering** model ya **two-tower embedding model**: ek tower user embeddings generate karta hai, doosra movie embeddings.
- Input layer: user ID, movie ID, aur contextual features (time of day, device, etc.)

- Hidden layers: dense layers jo user-movie interaction patterns learn karte hain.
- Output layer: predicted rating ya "probability of watching" score (sigmoid activation for binary, ya linear for rating prediction).

Stage 3: Training

- Loss function define karo (e.g., binary cross-entropy for click-prediction, MSE for rating prediction).
- Optimizer choose karo (Adam commonly used) aur learning rate tune karo.
- Batches mein train karo, multiple epochs ke through, GPU acceleration use karke (large-scale data ke liye essential).
- Regularization techniques (dropout, L2) use karo overfitting avoid karne ke liye.

Stage 4: Evaluation

- Validation set par metrics check karo: precision@K, recall@K, NDCG (ranking quality ke liye), ya RMSE (rating prediction ke liye).
- A/B testing offline metrics ke saath karo to ensure model real-world mein bhi accha perform karega.
- Edge cases test karo — naye users (cold-start problem), niche genres, etc.

Stage 5: Deployment

- Trained model ko serving infrastructure mein deploy karo (e.g., TensorFlow Serving, ONNX, ya cloud-based API endpoint).
- Real-time inference pipeline banao jo live user activity ke basis par recommendations fetch kare.
- Monitoring set karo: model drift detect karo, click-through rate track karo, periodically retrain karo naye data ke saath taaki recommendations fresh aur relevant rahein.
- Feedback loop banao — jo recommendations clicked/watched hue unko future training data mein feed karo (continuous improvement).